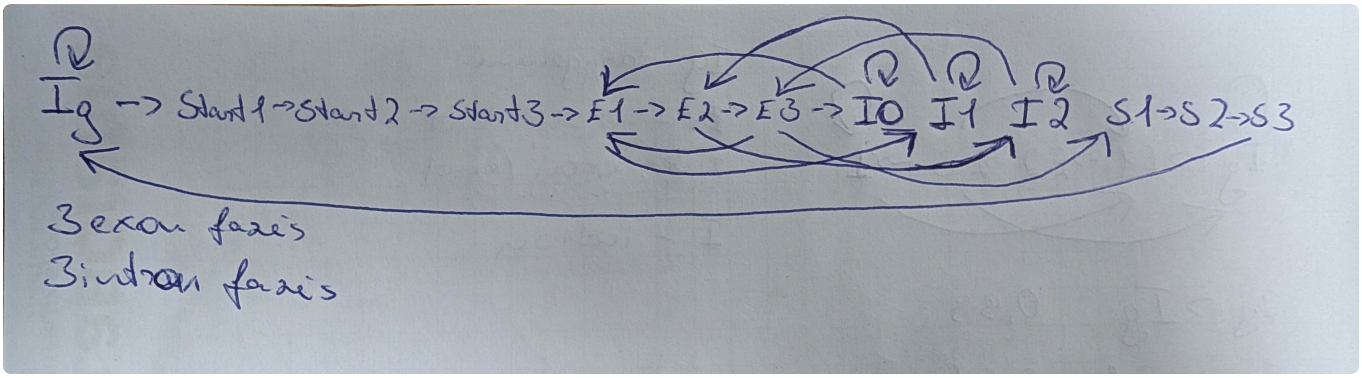
HMM

I wrote this in md, so sorry if it looks weird.

Suggested HMM model

My suggested HMM model. I believe it fixes the problem suggested in the slides. Because I add 3 intron phases.



States

```
states = ["ig", "start1", "start2", "start3", "e1", "e2", "e3", "i0", "i1",
"i2", "s1", "s2", "s3"]
```

`Ig` state represents the intergenic region between genes.

The `start1`, `start2` and `start3` states represent the three nucleotides of the start codon ATG.

The `e1`, `e2` and `e3` states represent the first, second and third positions of each codon in the coding region. These states repeat as the model moves through the sequence.

The `i0`, `i1` and `i2` states represent introns with different phases:

- Phase 0: the intron occurs between two complete codons.
- Phase 1: the intron occurs after the first nucleotide of a codon.
- Phase 2: the intron occurs after the second nucleotide of a codon.

`s1`, `s2` and `s3` represent the three positions of a stop codon.

Transitions

Transition table could look like this:

```
transitions = {
        "ig": {"ig": 0.99, "start1": 0.01},

        "start1": {"start2": 1.00},

        "start2": {"start3": 1.00},

        "start3": {"e1": 1.00},

        "e1": {"e2": 0.95, "i1": 0.05},

        "e2": {"e3": 0.95, "i2": 0.05},

        "e3": {"e1": 0.84, "i0": 0.05, "s1": 0.11},

        "i0": {"i0": 0.99, "e1": 0.01},

        "i1": {"i1": 0.99, "e2": 0.01},

        "i2": {"i2": 0.99, "e3": 0.01},

        "s1": {"s2": 1.00},

        "s2": {"s3": 1.00},

        "s3": {"ig": 1.00}
    }
```

Model can remain in `Ig` or move to `start1`. High probability to remain in `Ig`, because intergenic region can contain nucleotides.

The transitions from `start1` to `start2` and from `start2` to `start3` have a probability of 1.00 because all three positions are needed to complete the start codon. After this, the model moves to `e1`, which represents the first position of the next codon.

In the coding region, the usual order is `e1 → e2 → e3 → e1`. However, the model can also enter an intron:

- `e1 → i1 → e2`: the intron interrupts the codon after its first nucleotide.
- `e2 → i2 → e3`: the intron interrupts the codon after its second nucleotide.
- `e3 → i0 → e1`: the intron occurs between complete codons.

For `e3`, I divided the probability between three possible events: continuing to the next codon (0.84), entering a phase 0 intron (0.05), and entering the stop codon states (0.11). They allow

the model to continue coding most of the time, while also allowing introns and gene termination. Values are random.

Each intron state can repeat, allowing introns of different lengths. When the model leaves an intron, it returns to the correct coding position. For example, `i1` can return only to `e2`, because the first nucleotide of that codon has already been read.

The transition to `s1` is allowed only from `e3`. This means that the stop codon starts after a complete coding codon. After passing through the three stop states, the model returns to the intergenic region.

Emissions

Emissions like this:

```
emissions = {

        "ig": {"A": 0.30, "C": 0.20, "G": 0.20, "T": 0.30},

        "start1": {"A": 1.00},

        "start2": {"T": 1.00},

        "start3": {"G": 1.00},

        "e1": {"A": 0.20, "C": 0.30, "G": 0.30, "T": 0.20},

        "e2": {"A": 0.25, "C": 0.25, "G": 0.25, "T": 0.25},

        "e3": {"A": 0.20, "C": 0.20, "G": 0.40, "T": 0.20},

        "i0": {"A": 0.30, "C": 0.20, "G": 0.20, "T": 0.30},

        "i1": {"A": 0.30, "C": 0.20, "G": 0.20, "T": 0.30},

        "i2": {"A": 0.30, "C": 0.20, "G": 0.20, "T": 0.30},

        "s1": {"T": 1.00},

        "s2": {"A": 0.70, "G": 0.30},

        "s3": {"A": 0.70, "G": 0.30}
    }
```

Emission probabilities describe how likely each nucleotide is in a particular state. The emission probabilities in each state sum to 1.

For the intergenic and intron states, I assigned slightly higher probabilities to A and T. This represents an assumed preference for AT-rich sequences in this example. It is not a rule for every organism. (It was an AI suggestion to make it like that).

The start states emit A, T and G with a probability of 1.00. Therefore, this model allows only ATG as the start codon. I am not sure if this is the correct way of doing it.

For `e1`, I assigned C and G a probability of 0.30 each, and A and T a probability of 0.20 each. This gives the first coding position a slight GC preference.
For `e2`, all four nucleotides have a probability of 0.25. I used equal probabilities as a simple assumption because I did not estimate a nucleotide preference for this position.
For `e3`, G has a probability of 0.40, while the other nucleotides have a probability of 0.20 each. This makes G twice as likely as each other nucleotide at the third codon position.

The intron states have identical emission probabilities because their main purpose is to remember the coding position. They do not represent three different types of nucleotide composition.

The stop states are intended to allow TAA, TAG and TGA. However, the current emissions also allow TGG because `s3` does not remember which nucleotide was emitted by `s2`. This is kinda of a problem. Don't know how to solve it.

For the stop states, I assigned T a probability of 1.00 in `s1`, and A and G probabilities of 0.70 and 0.30 in both `s2` and `s3`. This makes A more likely than G at these positions. However, these independent emissions produce TAA with probability 0.49, TAG with 0.21, TGA with 0.21, and TGG with 0.09.

Example

`CCC | ATG | G | GTCCAG | CT | AAA | TAA | CCC`

The corresponding state path would be:

`ig ig ig | start1 start2 start3 | e1 | i1 i1 i1 i1 i1 i1 | e2 e3 | e1 e2 e3 | s1 s2 s3 | ig ig ig`

In this example, the intron interrupts the codon after G, its first nucleotide. The model enters `i1`, reads the intron and then returns to `e2` to read C, followed by `e3` to read T.

After removing the intron, the coding sequence would be:

`ATG | GCT | AAA | TAA`

Disclaimer

I used AI to properly recognize the problem suggested in the slides. I did not use it to create this model, just to understand intron phases. (Used model GPT 6.1 sol light)